

PyCore

Execution Flow Specification

From Python source through CPython 3.14 image construction to multi-cycle bytecode retirement on the SystemVerilog core.

Document type	Normative technical specification
Scope	Host tooling + RTL control/datapath
ISA	CPython 3.14 bytecode subset
Value model	Tagged 132-bit entries {tag[3:0], value[127:0]}
Primary path	image_from_source.py → pycore_system
Companion docs	architecture.md, bytecode_support.md, preprocessing_breakdown.md

Normative RTL: pycore/rtl/

Normative tooling: pycore/tools/image_from_source.py

Fidelity rule: 1:1 CPython code-unit order; no opcode rewrite.

Version aligned to repository `main` / PyCore image-boot path

July 13, 2026

Contents

1	Purpose and Normative Scope	2
2	End-to-End Pipeline Overview	2
3	Host-Side Image Construction	2
3.1	Tool chain roles	3
3.2	Builder algorithm	3
3.3	Instruction memory slot encoding	3
3.4	Object-graph serialization	4
3.5	Worked host example	4
4	System Topology	5
4.1	Memory map (data memory)	5
5	Tagged Value Model	6
6	Register File and Operand Stack	7
7	Control FSM	7
7.1	State responsibilities	7
8	Fetch Stage	8
9	Decode Stage	8
10	Execute Fabric	9
11	Container Operations	10
11.1	Shared phase machine	10
11.2	Heap bump allocator	10
11.3	In-memory layouts	10
11.4	Operations routed to S_CONTAINER	11
12	Calls, Frames, and Returns	11
12.1	Interim function model	11
12.2	CPython 3.14 CALL stack layout	11
13	Boot Sequence	11
14	Trap Architecture	12
15	Instruction Lifecycle Timeline	12
16	End-to-End Worked Trace	12
16.1	Per-opcode micro-flows (selected)	14
17	Bytecode Support Matrix (Summary)	14
18	Metrics	14
19	Design Constraints and Non-Goals	15
20	Reader Checklist	15

21 Document Control**15**

1 Purpose and Normative Scope

This document specifies the complete execution path of a Python program on **PyCore**: a research CPU whose native ISA is a subset of CPython 3.14 bytecode. It is written so that a reader unfamiliar with the repository can reconstruct, from this specification alone:

1. how host-side tooling turns `.py` source into memory images;
2. how those images are laid out in instruction and data memory;
3. how reset bootstraps a module code object and globals dictionary;
4. how each bytecode walks the multi-cycle control FSM to retirement;
5. how tags, traps, frames, and containers interact with that FSM.

Normative sources of truth

- **RTL**: `pycore_defs.svh`, `pycore_core.sv`, `pycore_fetch.sv`, `pycore_decode.sv`, `pycore_exec.sv`, `pycore_frame.sv`, `pycore_system.sv`.
- **Image builder**: `pycore/tools/image_from_source.py` (primary). `preprocess.py` is legacy and out of scope for new image-boot programs.
- **Encoding helpers**: `encoding.py`, `heap_image.py`.

Fidelity boundary

“Identical to what a CPython compiler would create” means: the image contains the same object graph as `compile()` output—same bytecode units in the same order (including **CACHE** and **EXTENDED_ARG**), same `co_consts/co_names`, and nested code objects—lowered mechanically into tagged 128-bit-slot encoding. *No opcode is added, removed, reordered, rewritten, or argument-remapped.* Byte-exact CPython C-struct layout is out of scope.

2 End-to-End Pipeline Overview

Figure 1 is the master flow. Everything else in this document refines one stage of this pipeline.

Operational contract

1. Host tooling *must* run under Python 3.14 (hard version gate).
2. Unsupported / deferred opcodes *must* be rejected before image emission when possible.
3. Hardware treats any remaining illegal opcode as `PY_TRAP_ILLEGAL_OPCODE` and enters `S_HALT`.
4. There is no software fallback interpreter on-core.

3 Host-Side Image Construction

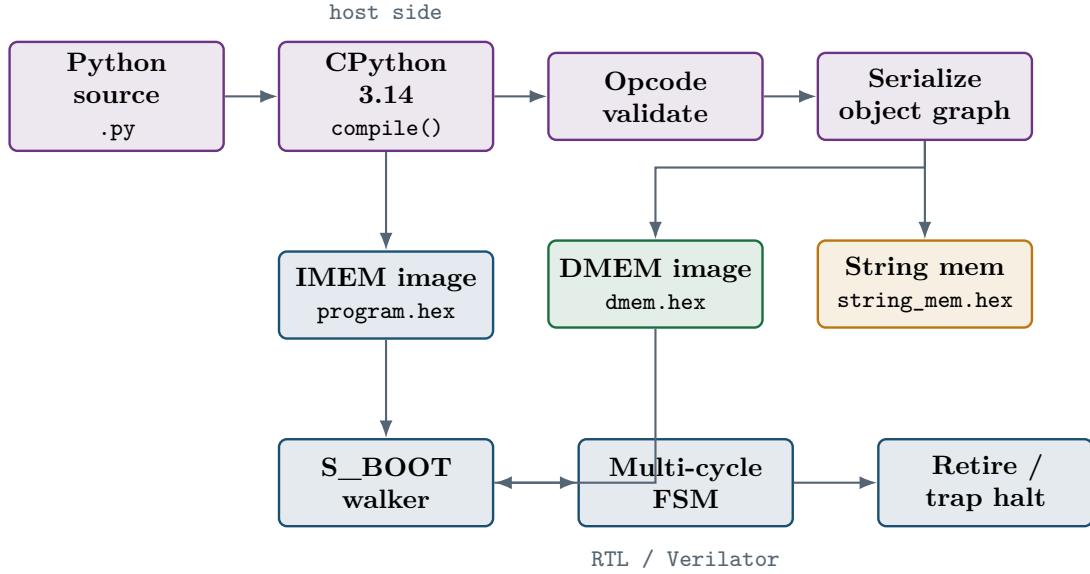


Figure 1: Master pipeline from Python source to instruction retirement.

3.1 Tool chain roles

Module	Role	Normative behavior
image_from_source.py	Primary builder	<code>compile</code> → <code>validate</code> → 1:1 imem transcode → <code>serialize heap</code> → <code>boot record</code>
encoding.py	Tag codec	Mirrors <code>pycore_defs.svh</code> tags; short/long string packing; imem slot formatting
heap_image.py	DMEM serializer	LIST/DICT/TUPLE/code-object layouts identical to RTL helpers
run_image_test.py	Differential harness	Runs <code>managed_entry()</code> on host; writes expected tag/value into meta
preprocess.py	<i>Legacy</i>	Branch remap / CACHE strip / inline LOAD_CONST ; not for new image-boot tests

3.2 Builder algorithm

Figure 2 details the host algorithm implemented by `image_from_source.py`.

3.3 Instruction memory slot encoding

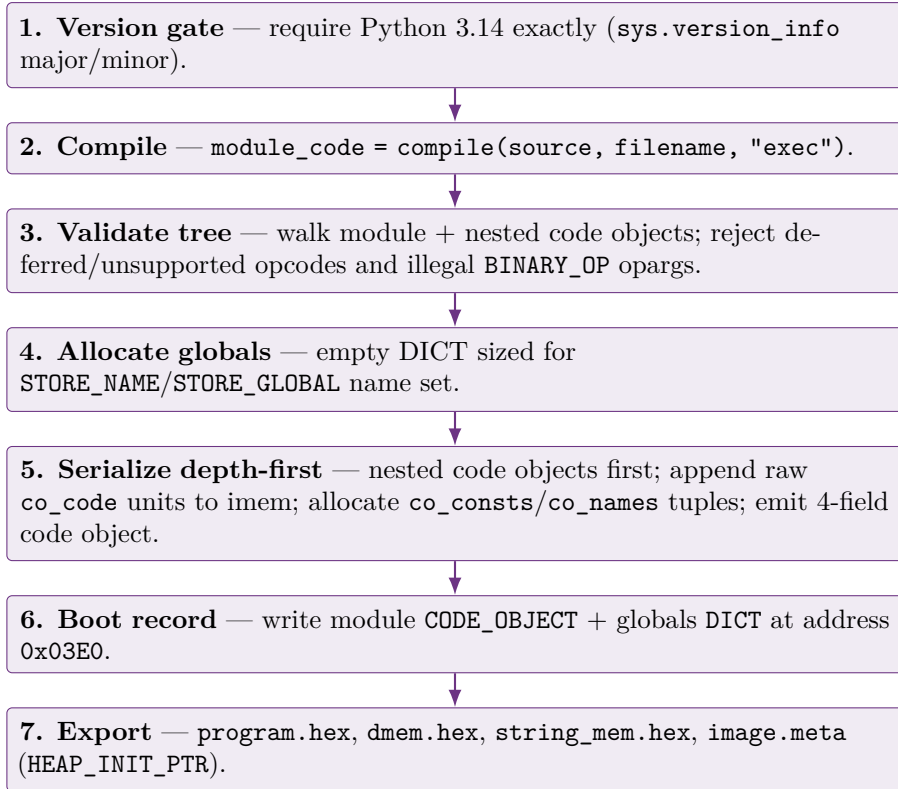
Each CPython two-byte code unit (`opcode`, `arg8`) becomes one 64-bit imem slot:

$$\text{slot} = \{ 0^{24}, \text{arg}[31:0], \text{opcode}[7:0] \} \quad \text{stored as 16 hex digits.} \quad (1)$$

In practice `format_imem_slot` packs `bits[39:8]=arg`, `bits[7:0]=opcode`. Slot index i equals CPython code-unit index i ; relative jump arguments are therefore the compiler's original arguments.

CACHE and EXTENDED_ARG policy

Both remain *present* in the image. Fetch folds **EXTENDED_ARG** streams and skips **CACHE** units at execution time. Preprocessing must not strip them on the image-boot path.

Figure 2: Host image-builder control flow (`image_from_source.py`).

3.4 Object-graph serialization

Constants are lowered by tag:

Python value	Tag	Value encoding
<code>int</code> (fast path)	INT	sign-extended / masked to 128 bits
<code>float</code>	FLOAT	IEEE 754 binary64 in <code>value[63:0]</code>
<code>bool</code>	BOOL	<code>value[0]</code>
<code>None</code>	NONE	0
<code>str</code> ≤ 15 B UTF-8	SHORT_STR	size+payload+flags inline
<code>str</code> > 15 B	LONG_STR	{size, addr} + string mem
<code>tuple</code>	TUPLE	{size, addr} element array
<code>code</code>	CODE_OBJECT	4-field descriptor (below)

Code-object fields (32 bytes each = value slot + tag slot):

Field	Tag	Contents
0	INT	<code>entry_slot</code> (imem slot index)
1	TUPLE	<code>co_consts</code> handle
2	TUPLE	<code>co_names</code> handle
3	INT	packed {stacksize, nlocals, argcount}

3.5 Worked host example

Consider the minimal program:

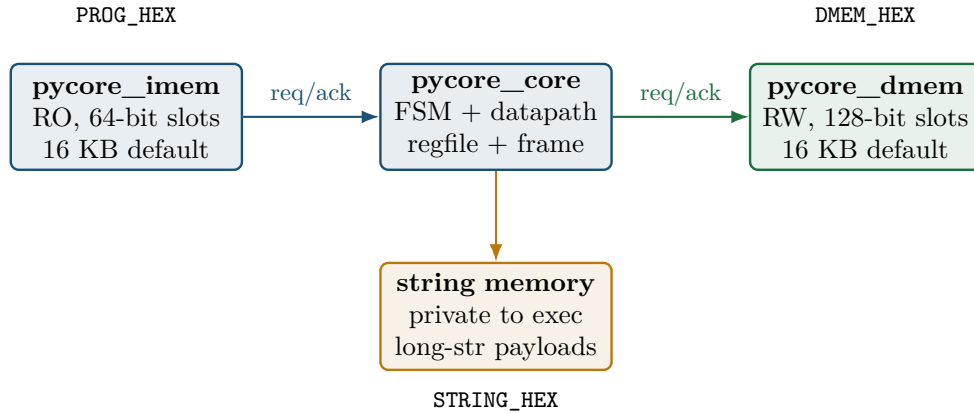
Listing 1: pycore/programs/smoke_return.py

```
def managed_entry() -> int:
    return 12
```

1. Module bytecode defines `managed_entry`, stores it into globals, then typically loads and calls it (exact module prologue depends on CPython 3.14 wordcode for the module body).
2. Nested function code object is serialized first: its `co_code` is appended to `imem`; `co_consts` contains the integer 12.
3. Module code object follows; globals DICT is empty at image time and filled by runtime `STORE_NAME`.
4. Boot record points at the *module* code object and the globals DICT.

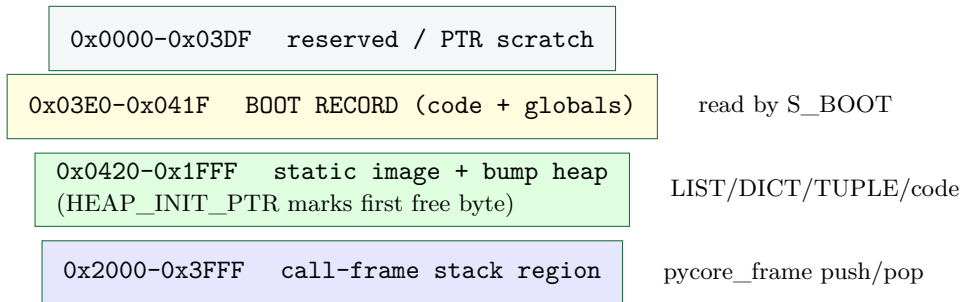
4 System Topology

PyCore is a Harvard machine. The simulation top `pycore_system` wires the core as a memory master to separate `imem` and `dmem` banks (Figure 3).

Figure 3: Harvard system topology (`pycore_system`).

4.1 Memory map (data memory)

Default parameters: `ADDR_WIDTH=32`, `BLOCK_SHIFT=12`, 4 `imem` blocks + 4 `dmem` blocks (16 KB each).

Figure 4: DMEM address map (normative defaults from `pycore_defs.svh`).

Boot record layout

At PYCORE_BOOT_RECORD_ADDR = 0x03E0:

0x3E0: module code object value

0x3F0: module code object tag

0x400: globals dict value

0x410: globals dict tag

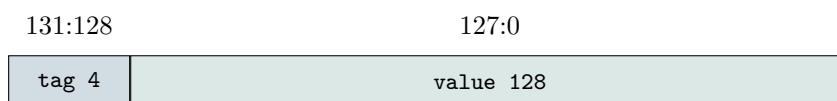
Static heap allocations begin at $\max(\text{HEAP_BASE}, \text{BOOT_RECORD_ADDR} + 64)$ so they do not collide with the boot record.

5 Tagged Value Model

Every architectural value is a 132-bit register-file entry:

$$E = \{ \text{tag}[3:0], \text{value}[127:0] \}. \quad (2)$$

When stored in DMEM, E is split across two consecutive 16-byte slots (value first, then tag in [3:0] of the second slot). This keeps all dmem addressing 16-byte aligned.



INT: value[63:0] signed fast path, SEXT to 128
 FLOAT: IEEE754 binary64 in value[63:0]
 SHORT_STR: size[127:124], bytes[123:4], flags[3:0]
 LONG_STR / TUPLE: {size[127:64], addr[63:0]}
 LIST / DICT / CODE_OBJECT: addr in low bits

Figure 5: Tagged entry bit layout.

Code	Tag	Meaning
0000	UNINITIALIZED	Undefined local; reads trap
0001	INT	Signed 64-bit fast path
0010	FLOAT	IEEE 754 double
0011	BOOL	Truth in <code>value[0]</code>
0100	PTR	128-bit byte address (v1 uses low 32)
0101	TUPLE	{size, addr}
0110	SHORT_STR	Inline ≤ 15 UTF-8 bytes
0111	LONG_STR	External string-memory descriptor
1000	OBJECT	Opaque / unsafe; traps on ops
1001	DICT	Heap dict handle
1010	LIST	Heap list handle
1011	SET	Reserved (deferred)
1100	CODE_OBJECT	Callable code descriptor
1101	FRAME_OBJECT	Reserved frame object tag
1110	NULL	CPython <code>self_or_null</code> sentinel
1111	NONE	Python None

6 Register File and Operand Stack

`pycore_regfile` provides 96 architectural entries:



Figure 6: Architectural register-file windows.

The operand-stack pointer `tos` is a core register updated in `S_WB` (or inside `S_CONTAINER` for container ops). Decode reads `tos` combinationally; because the machine is non-pipelined, no operand forwarding is required.

7 Control FSM

The core is multi-cycle and non-pipelined: exactly one instruction is in flight. Figure 7 is the normative state machine from `pycore_core.sv`.

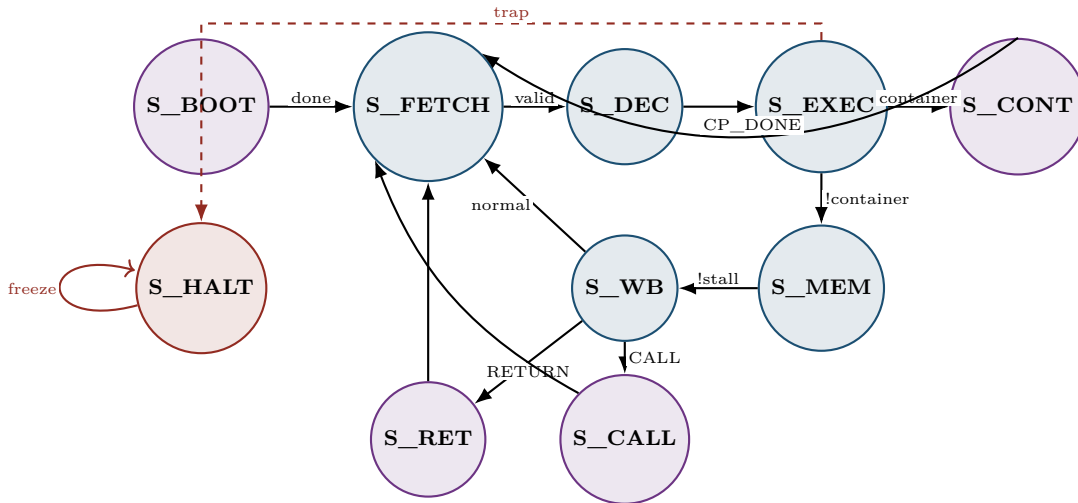


Figure 7: PyCore control FSM (states numbered 0–9 in RTL).

7.1 State responsibilities

`S_BOOT`

When `BOOT_EN=1`, read boot record; cache `co_consts/co_names`; latch `globals_base`; redirect fetch to module entry slot.

`S_FETCH`

Run `pycore_fetch` until a real instruction is presented; latch `{opcode, arg, pc}`. Fold `EXTENDED_ARG`; skip `CACHE`.

S_DECODE

Drive RF read addresses; latch operands.

S_EXEC

Execute fabric / branch unit; may stall on multi-cycle units. If `dec_is_container`, divert to **S_CONTAINER**.

S_MEM PTR load/store via `pycore_mem_stage`; hold on outstanding dmem access.

S_WB Write RF; advance `tos`; redirect fetch on taken branch; enter **S_CALL**/**S_RETURN** when indicated.

S_CONTAINER

Multi-cycle container / name / const path; bypasses **S_MEM**/**S_WB**.

S_CALL

/ **S_RETURN** Frame push/pop + code-object field reload.

S_HALT

Terminal freeze on any trap.

Hazard rule

Because only one instruction is ever in flight, there are **no pipeline hazards**. Multi-cycle execute or memory simply holds the FSM in **S_EXEC**/**S_MEM** until completion.

8 Fetch Stage

`pycore_fetch` is a registered req/ack imem master. Address formation:

$$\text{imem_addr} = \text{pc} \ll 3 \quad (8\text{-byte slots}). \quad (3)$$

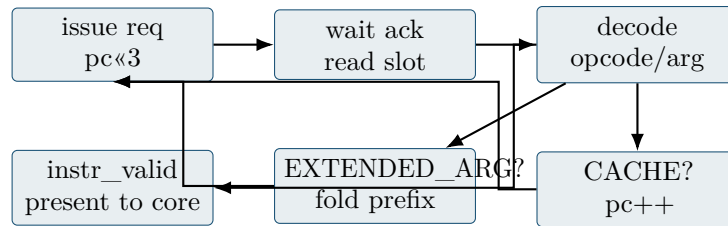


Figure 8: Fetch internal behavior: fold, skip, or present.

Argument folding:

$$\text{arg}_{\text{eff}} = \begin{cases} (\text{prefix} \ll 8) \mid \text{arg}[7:0] & \text{if prior EXTENDED_ARG} \\ \text{arg} & \text{otherwise.} \end{cases} \quad (4)$$

The core freezes fetch via `stall` while an instruction is in later states, and ignores stale `instr_valid` until a fresh fetch completes after re-entering **S_FETCH**.

9 Decode Stage

`pycore_decode` is combinational. It maps (`opcode`, `arg`, `tos`, `locals_base`) to:

- ALU op / branch / call / return / container class bits;

- RF selectors `rs1/rs2/rd`;
- stack push/pop effects;
- memory op class (`LOAD_FAST/STORE_FAST/PTR`).

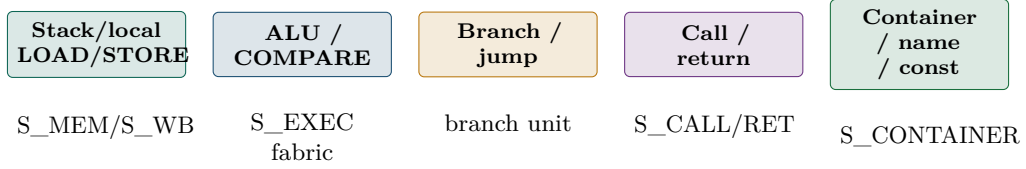


Figure 9: Decode classification → FSM diversion.

Notable CPython 3.14 decode rules baked into RTL:

- **COMPARE_OP**: selector is `oparg` $\gg$ 5 (range 0..5).
- **LOAD_GLOBAL**: `namei` = `oparg` $\gg$ 1; if `oparg` & 1, also push NULL.
- **BINARY_OP** with `oparg` `NB_SUBSCR`=26 sets `PY_ALU_SUBSCR` and routes to `S_CONTAINER`.
- Relative jumps: `target` = `pc` + 1 + $n_{\text{cache}} \pm \text{arg}$ with inline-cache counts from CPython 3.14.

10 Execute Fabric

The execute stage is a tag-routed fabric (Figure 10).

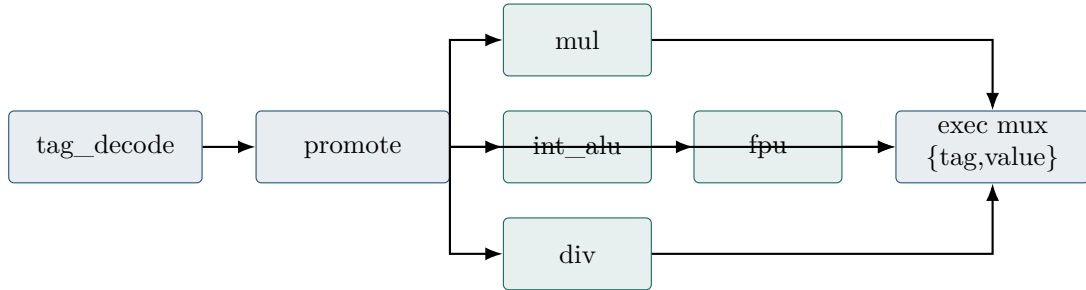


Figure 10: Tag-routed execute fabric.

Promotion rules selected by tag decode:

Promotion	When
<code>BOOL</code> → <code>INT</code>	integer ALU path needs numeric
<code>BOOL</code> → <code>FLOAT</code> / <code>INT</code> → <code>FLOAT</code>	true divide or float op
<code>NB_TRUE_DIVIDE</code>	always yields <code>FLOAT</code>

Semantic deviation — integers

Integer add/sub/mul/shift wrap at 64 bits. This is the largest intentional deviation from CPython arbitrary-precision integers. Programs outside signed 64-bit range can silently diverge unless rejected earlier.

String **BINARY_OP** (+) concatenates `SHORT_STR/LONG_STR` pairs inside the exec unit's private string memory; oversized results raise `MEM_FAULT`.

11 Container Operations

S_CONTAINER handles multi-cycle heap and name operations, entered directly from S_EXEC when `dec_is_container` is set. It bypasses S_MEM and S_WB; TOS/RF updates occur inside the container FSM.

11.1 Shared phase machine

Phase	Name	Purpose
0	CP_INIT	First active cycle; launch first dmem/RF op
1	CP_HDR	Header R/W in flight
2	CP_VAL	Element value R/W in flight
3	CP_TAG	Element tag R/W in flight
4	CP_DONE	Terminal; combinational return to S_FETCH

Dict-specific phases extend through CP_DICT_HASH..CP_DICT_RD_VTAG for probe/hash sequences.

11.2 Heap bump allocator

$$\text{HEAP_BASE} = 0 \times 0400, \quad \text{HEAP_LIMIT} = 0 \times 2000, \quad \text{capacity} \approx 7 \text{ KB}. \quad (5)$$

`heap_ptr_r` advances monotonically from `HEAP_INIT_PTR` (from image meta). Overflow $\Rightarrow$ `PY_TRAP_MEM_FAULT`. No free list / GC in this prototype.

11.3 In-memory layouts

LIST at base	DICT at base
hdr {capacity, length}	hdr {slot_count, used}
elem[0] value	key value
elem[0] tag	key tag
elem[i] ... (stride 32 B)	val value
	val tag (stride 64 B)

TUPLE: no header; size lives in the handle. Element i at `addr + 32i` (value) / `+16` (tag).
STORE_SUBSCR on TUPLE traps TYPE.

Figure 11: Container DMEM layouts (16-byte slot granularity).

Dict hashing (open-addressed linear probe, tombstone-free):

Key tag	Hash
INT / BOOL	value[31:0]
SHORT_STR	XOR of four 32-bit words of value
LONG_STR	value[31:0] $\oplus$ value[95:64]

Slot count at construction: `next_pow2(max(4, 2 ·  $n_{\text{pairs}}$ ))` (load $\leq 50\%$). Empty-bucket sentinel: key tag `UNINIT`. Inserts that would fill the table completely trap so probes always terminate.

11.4 Operations routed to S_CONTAINER

Bytecode	Container op	Behavior
BUILD_LIST	CONT_BUILD_LIST	bump-alloc header+elements; push LIST
BUILD_TUPLE	CONT_BUILD_TUPLE	alloc elements only; push TUPLE
BUILD_MAP	CONT_BUILD_MAP	alloc+probe-insert pairs; push DICT
BINARY_OP NB_SUBSCR	CONT_SUBSCR_*	list/tuple index or dict probe
STORE_SUBSCR	CONT_STORE_*	list write or dict upsert
LOAD_CONST	CONT_LOAD_CONST	two dmem reads from <code>co_consts</code>
LOAD_GLOBAL/LOAD_NAME	name lookup	<code>co_names</code> then globals probe
STORE_NAME/STORE_GLOBAL	globals store	upsert into module DICT

12 Calls, Frames, and Returns

12.1 Interim function model

function $\equiv$ **CODE_OBJECT**

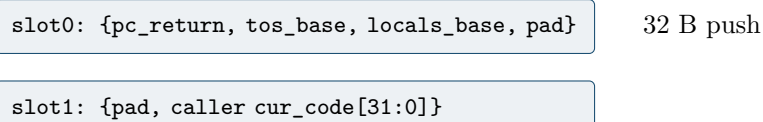
MAKE_FUNCTION checks TOS is **CODE_OBJECT** and leaves it in place. Defaults, annotations, closures, and bound methods are rejected by tooling / unsupported in hardware.

12.2 CPython 3.14 CALL stack layout

Bottom $\rightarrow$ top before **CALL**:

$$[\text{callable}, \text{NULL}, \text{arg}_1, \dots, \text{arg}_N] \quad (6)$$

Non-method calls require sentinel tag **NULL**. Hardware validates callable tag and **argcount**, then enters the frame manager.



On RETURN: pop slot1 then slot0; restore caller code pointer, PC, TOS/locals bases; reload `co_consts/co_names`; deposit return value; resume fetch.

Figure 12: Call-frame descriptor pushed by `pycore_frame`.

Frame depth is bounded by the reserved region `0x2000–0x3FFF`. A ring-buffer spill design exists only in `rtl/attic/` and is *not* on the production path.

13 Boot Sequence

BOOT_EN=0 remains only for legacy hand-authored hex fixtures that fetch from PC 0 without a boot record.

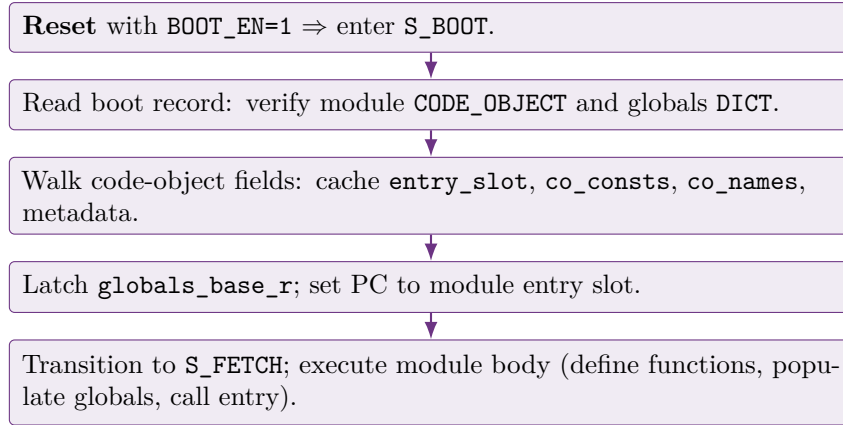


Figure 13: Image-boot walker.

14 Trap Architecture

`pycore_trap` latches a trap and forces `S_HALT`, freezing architectural state and the cycle counter. Priority (highest first):

Pri.	Code	Cause
1	TYPE (1)	Illegal tag for arithmetic/compare/branch/unary; bad dict key tag; store into tuple
2	STACK (2)	Operand-stack overflow/underflow at WB
3	DIV_ZERO (3)	Integer division by zero
4	FPU_EXCEPTION (4)	Floating-point exception leaf
5	ILLEGAL_OPCODE (5)	Decode miss / unsupported opcode
6	CALL_FILTER (6)	Non-callable, bad argc, frame-manager fault
7	ADDR_ALIGN (8)	Misaligned dmem access (raised in <code>S_MEM</code>)
8	MEM_FAULT (7)	OOR address, missing global/key, bad PTR, heap OOM

No exception objects

Missing dict keys and undefined globals raise `MEM_FAULT`, not Python `KeyError`/`NameError` objects. There is no on-core exception object protocol in this prototype.

15 Instruction Lifecycle Timeline

Figure 14 shows a generic non-container ALU instruction.

Container ops replace the MEM/WB tail with an internal phase loop:

$$S_FETCH \rightarrow S_DECODE \rightarrow S_EXEC \rightarrow S_CONTAINER(CP_*) \rightarrow S_FETCH. \quad (7)$$

16 End-to-End Worked Trace

This section walks a concrete path for a module that defines and calls a function returning a small integer (pattern of `smoke_return.py` and `img_smoke.py`).

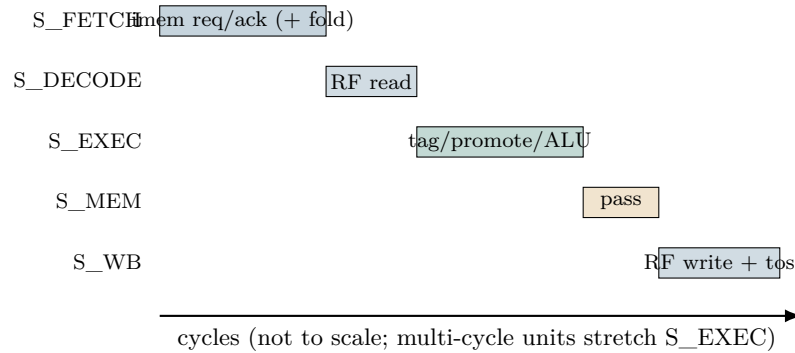


Figure 14: Nominal lifecycle of a simple ALU bytecode.

Host: compile module; validate; serialize nested code then module; write boot record.
Images: imem gets raw units; dmem gets code objects, consts tuple containing INT 12, empty globals DICT.
S_BOOT: load module CODE_OBJECT + globals; PC ← module entry
Module body: MAKE_FUNCTION / STORE_NAME populates globals; LOAD_NAME+PUSH_NULL+CALL
Callee: LOAD_CONST 12 via S_CONTAINER; RETURN_VALUE pops frame and writes INT 12 to caller TOS.

Figure 15: Narrative trace from source to returned INT.

16.1 Per-opcode micro-flows (selected)

LOAD_CONST. Decode sets `is_container`. `S_CONTAINER` indexes `co_consts[arg]`, performs value then tag dmem reads, pushes the tagged entry, returns to fetch.

LOAD_GLOBAL / LOAD_NAME. Read name string from `co_names`; probe module globals DICT; push value; optionally push NULL when `oparg&1`. Missing name $\Rightarrow$ `MEM_FAULT`. No builtins fallback.

CALL. Validate callable+NULL+argc; read callee code-object fields; push frame; transfer locals/args; redirect fetch to callee `entry_slot`.

RETURN_VALUE. Pop frame; restore caller context; write return entry to caller stack; fetch resumes at return PC. Base-frame return completes the program (testbench observes retired value).

BINARY_OP (arithmetic). Pop two stack entries; tag-decode + promote; run int/float unit; push result tag/value; wrap on 64-bit int overflow.

BUILD_LIST / NB_SUBSCR. Bump-allocate or index through `S_CONTAINER`. Negative indices do not wrap; out-of-range $\Rightarrow$ `MEM_FAULT`.

17 Bytecode Support Matrix (Summary)

Full detail lives in `bytecode_support.md`. Summary classes:

Class	Members (representative)
Fully supported	CACHE, EXTENDED_ARG, RESUME, LOAD_FAST*, STORE_FAST, LOAD_SMALL_INT, LOAD_CONST, LOAD_GLOBAL, LOAD_NAME, STORE_NAME/GLOBAL, PUSH_NULL, MAKE_FUNCTION, CALL, TO_BOOL, NOT_TAKEN, POP_TOP/POP_ITER, RETURN_VALUE, JUMP_*, POP_JUMP_IF_*, BUILD_LIST/TUPLE/MAP, BINARY_OP (supported opargs incl. NB_SUBSCR), STORE_SUBSCR, COMPARE_OP (0..5)
Partial	LOAD_FAST_BORROW LOAD_FAST_BORROW (container two-beat / legacy expand), BINARY_OP variants, COPY/SWAP (rejected by builder), OR_POP jumps
Deferred	LIST_APPEND, MAP_ADD, LIST_EXTEND, DICT_UPDATE/MERGE, DELETE_SUBSCR, CONTAINS_OP, slices, BUILD_SET / SET_*
Unsupported	Any other CPython 3.14 opcode: builder reject or ILLEGAL_OPCODE trap

18 Metrics

$$\text{CPO} = \frac{\text{total_cycles}}{\text{dynamic_opcodes}} \quad (8)$$

`cycle_count` increments every non-reset, non-trapped cycle. Secondary metrics: type-trap rate and unit utilization. Helper `pycore/tools/cosim_trace.py` summarizes traces containing `opcode=`, `unit=`, and `trap=` fields.

19 Design Constraints and Non-Goals

1. **No hidden software runtime in preprocessing.** Host tooling is a thin validate/transcode layer only.
2. **No CPython C-struct fidelity.** Tagged slots are required for hardware access.
3. **No full object protocol.** `OBJECT` means unsafe/trap, not a slow path.
4. **No builtins chain.** Name lookup is module globals only.
5. **No GC.** Bump heap only.
6. **No pipelining.** Correctness via single-issue multi-cycle FSM.

Known semantic deviations

INT/BOOL dict-key non-equivalence; missing key $\rightarrow$ `MEM_FAULT`; unsigned index checks; `LONG_STR` dict equality is descriptor equality relying on interning; no rehash/grow; 64-bit int wrap; function $\equiv$ code object. See `bytecode_support.md` §“Semantic deviations from CPython”.

20 Reader Checklist

A reader who has absorbed this specification should be able to:

1. Point to the host function that turns `.py` into hex images and state the fidelity rule it obeys.
2. Draw the `DMEM` map and locate the boot record, heap, and frame stack.
3. Explain why `CACHE` appears in `imem` but never reaches `decode`.
4. Trace `LOAD_CONST` through `S_CONTAINER` rather than an inline immediate encoding.
5. Describe `CALL`’s stack layout and the two-slot frame descriptor.
6. State trap priority and which faults freeze the machine into `S_HALT`.
7. Identify which container layouts use headers (`LIST/DICT`) versus inline size (`TUPLE`).

21 Document Control

Field	Value
Path	<code>pycore/docs/pycore_execution_flow.tex</code>
Build	<code>pdflatex pycore_execution_flow.tex</code> (run twice for ToC)
Related	<code>architecture.md</code> , <code>bytecode_support.md</code> , <code>preprocessing_breakdown.md</code>
Status	Living specification; update when FSM, image format, or tag map changes